Replicating Marin's optimizer tuning and transfer

Reference BO, multiplier grids, local optima, law fitting and honest holdouts

What Marin actually searches

Your recollection is right for the MoE scaling work: Marin searches **grids of multipliers around a heuristic**, estimates local LR optima, and refits the heuristic across width and token horizon. But its reference-model tuning also uses **Vizier Bayesian optimization**, and its systems sweeps optimize tokens/s rather than validation loss. These are three distinct procedures.

All code locators below refer to commit `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`; the evidence bundle includes source copies and complete issue/comment snapshots retrieved 2026-11-02. Issue results are publicly reported experiment evidence; we have not independently rerun the jobs or verified every W&B terminal.

Procedure	Primary artifact	What it determines
Reference tuning	`experiments/references/reference_hyperparameter_sweep.py`	A joint small-model optimizer configuration using Vizier
MoE LR surface	May #5951, August #7856	Local LR optima versus width and horizon, then a fitted transfer law
Recipe generation	`references/completed_adamh.py`, `grug/moe/heuristic.py`, `grug/moe_hero_ep/heuristic.py`	Deterministic full settings for a new model/token budget
Allocation/loss law	#8003	IsoFLOP size/token tradeoff using the fitted LR prescription
Systems tuning	`grug/moe_hero_fsdp/sweep.py`	Runtime/memory flags, not an optimizer-optimality claim

Reference-model code: 40 adaptive suggestions, not a full grid

The inspected `SWEEP` object selects approximately 130M Grug, a Nemotron/code/math mixture, context 4096, **64 sequences per update**, and **1B target tokens**. That implies 262,144 tokens/update and 3,814 whole updates, or 999,817,216 actual tokens after floor rounding. This particular file is a demonstration/current configuration, not proof that these were the exact settings of Delphi's published final reference (whose heuristic uses a 2.5B-token reference).

It runs ten suggest/train/update rounds, four suggestions per round: **40 proposed trials**. The objective is `eval/uncheatable_eval/macro_loss` minimized, with linear schedule, 10% warmup and a 20% decay setting. Study state is carried through a SQLite DB artifact; run artifacts preserve trial IDs and chosen parameters.

Parameter	Range in the inspected file
Matrix/Hyperball LR `lr`	5×10^{-5} to 0.03
Scalar Adam LR `adam_lr`	5×10^{-5} to 0.03
`beta1`, `beta2`	0.5 to 1.0
Epsilon	10^{-15} to 10^{-3}
Clip threshold	0.1 to 1.0
Logit z-loss coefficient	10^{-7} to 0.1

`run_vizier_suggest` calls `root.add_float_param(...)` without an explicit log scale and requests algorithm `DEFAULT`. Do not describe this as an exhaustive Cartesian product, or silently claim log-uniform sampling. For our adaptation, use explicit log coordinates for positive scales and exclude singular momentum endpoints, while documenting that this changes the search

setup.

`run_vizier_train` sets **both** LR groups, floor-rounds step count and logs tags. `run_vizier_update` reads the final metrics line, reports finite scores and marks NaN/Inf infeasible. That implementation deserves a replication check: ensure the final line really contains the requested **terminal, annealed evaluation**, rather than an earlier/best-so-far value. Failed jobs must be represented in the ledger; not every exception is automatically a completed infeasible trial.

May MoE sweep: explicit heuristic-relative grids

Issue #5951 records widths 512/768/1024/1280 and six active-TPP settings 4/10/20/60/120/240. The main LR multiplier grid is

$$\mathcal{M}_\eta = \{0.4, 0.7, 1.0, 1.3, 1.6\}.$$

Batch is fixed per width (32/64/128/256 sequences in the reported main groups). Additional long-horizon runs use active-TPP 5000 and a different LR grid. The fit uses **17 eligible local optima**, not every launched run. Boundary-censored bowls and incomplete cells were excluded, which means the fitted set has a selection mechanism that a replicator should retain and audit.

The issue documents a real accounting error: a fitting script assumed an FFN dimension $4d$ while the actual expert width was approximately $d/2$. This inflated the token inputs and LR coefficient. The corrected reported law is

$$\eta_H \approx 18.31 D^{-0.395} d^{-0.150} \sqrt{B_s},$$

where D is tokens, d width and B_s sequences at the fitted context 4096. The current May heuristic rewrites this using token batch $B_t = B_s \ell$:

$$\eta_{\mathrm{Adam}} = 0.06602 D^{-0.395} d^{-0.150} \sqrt{B_t}, \qquad \eta_H = \tfrac{13}{3} \eta_{\mathrm{Adam}},$$

with caps and additional epsilon/beta prescriptions. The coefficient conversion uses $\ell = 4096$. The earlier uncorrected issue-body law is not the final result; use the correction comment and pinned code.

August sweep: 150 proposed runs and a new LR surface

Issue #7856 specifies six widths 512/768/1024/1280/1536/2048, active-TPP settings 30/60/150/300/600, and

$$\mathcal{M}_\eta = \{0.7, 0.85, 1.0, 1.2, 1.4\},$$

for **150 proposed runs**, using a different architecture/mixture/context-8192 setup. The initial issue body says not launched; the August 5 update reports **122/150 at full budget**. Cite dated comments rather than treating the issue's opening status as current or claiming all 150 completed.

Local minima were estimated with quadratics in $\log \eta$. The reported law is

$$\eta_H \approx 34.35 D^{-0.346} d^{-0.345} \sqrt{B_s}, \qquad \ell = 8192,$$

with reported in-sample $R^2 \approx 0.978$. The **batch exponent is pinned at 1/2**, not learned from an exhaustive batch-size search. This does not determine an optimal batch-size law $B_\star(N, D)$. Our batch sweep remains necessary.

The inspected EP heuristic implements the equivalent token-batch form

$$\eta_{\text{Adam}} = 0.087571 D^{-0.3461} d^{-0.3448} \sqrt{B_t}, \qquad \eta_H = \tfrac{13}{3}\eta_{\text{Adam}},$$

plus LR caps, $\epsilon = 9.676 \times 10^{-18}\sqrt{D/B_t}$, and clipped $\beta_2 = 0.999^{B_t/131072}$. Separate momentum, epsilon and beta definitions are essential to replication. These values are **Marin recipe coefficients**, not new recommended AdamW settings for our corpus.

The TPP denominator changes the regime. The August grid's 30–600× values use active parameters; our earlier 200× target uses stored parameters. In general,

$$R_{\text{active}} = R_{\text{stored}} \frac{N_{\text{stored}}}{N_{\text{active}}}.$$

Using the launcher's approximate 535B/23B counts, 200 stored-TPP would correspond to roughly 4,650 active-TPP, whereas its 791 active-TPP target is roughly 34 stored-TPP. Thus that sweep is not direct validation of our more extreme stored-TPP target. The May issue's additional5000-active-TPP trials show the importance of checking long horizons, but their reported status and different recipe must be retained.

The issue also warns that extrapolation to width 6144/15T is far outside the observed range. Apparent agreement of old and new prescriptions near that one target can reflect cancellation between token and width exponents. A high R^2 inside the fit does not establish hero-scale reliability.

Optimizer groups and norm definitions are part of the algorithm

In `grug/moe_hero_ep/optimizer.py`, `GrugMoeMuonHConfig` assigns:

- **MuonH:** attention/expert/shared matrices and gated norms, with Newton–Schulz direction plus Frobenius projection.
- **AdamH:** output projection / LM head.
- **Adam:** token embeddings, router/bias, attention gates, 1-D norm gains and small SConv kernels.

For matrix W and direction U, the Hyperball construction takes a relative-norm step and projects back:

$$\widetilde{W} = W - \eta \frac{\|W\|_F}{\max(\|U\|_F, \epsilon_H)} U, \qquad W_{t+1} = \frac{\|W\|_F}{\max(\|\widetilde{W}\|_F, \epsilon_H)} \widetilde{W}.$$

The code distinguishes per-expert normalization from **one norm over a layer's expert bank**; changing the axes changes the optimizer. It includes sharding corrections before norm reduction to avoid overcounting a sharded matrix. An implementation with the same name but local-shard norms does not reproduce it.

Later production launchers also apply decay to particular Adam gate/router leaves and introduce trainer-level global clipping. This differs from saying "Hyperball means no parameter ever uses weight decay or clipping." Copy the optimizer **and launcher overrides** from the same version.

Why endpoint and boundary checks are mandatory

A constant-LR extension in #7856 initially chose the lowest tested multiplier in every completed budget. The investigators correctly treated that as a boundary result, then added lower values 0.10/0.20/0.32/0.45/0.70. The later comment reports an interior local optimum and worse constant-LR terminal loss than linear decay in that d512 experiment. This is evidence that schedule changes alter the useful LR bracket; it is not a universal theorem that one schedule dominates.

For each (d, D, B) cell, let $x = \log \eta$ and fit a diagnostic bowl

$$L(x) \approx ax^2 + bx + c, \qquad x_\star = -b/(2a).$$

Accept this interpolated optimum only if $a > 0$, the minimum lies inside the sampled interval, nearby observations support the curvature, and instability/seed noise do not dominate. Otherwise expand or record a bound. A fitted minimum is not an observed run; report both best tested loss and fitted loss.

Fit a global law only from eligible optima. Bootstrap independent training cells/seeds, validate leave-one-width and leave-one-horizon predictions, and reserve a fresh size/horizon for final evaluation. Keep logs from unstable and censored cells. This avoids a falsely clean frontier obtained by silently dropping difficult regimes.

An existing H100 reproduction entry point

The pinned repository already contains `experiments/grug/moe_hero_ep/launch_h100_scaling_ladder.py`. It defines d512 on 8 H100s and d768 on 16 H100s, both with global sequence batch 1024, 384 experts/top-8, half-width latent experts, and the hero's active-TPP prescription. Its comments give approximately 16B and 48B tokens respectively. This is a more direct first replication target than inventing a new PyTorch model immediately.

The H100 launcher selects `gpu_fa4_cute` attention and `fixed_pooled_wave_all_to_all` transport; the larger hero config uses an `sm100`-specific attention variant and a different default transport. Use the intended hardware launcher and qualify its dependencies, rather than copying one model object while ignoring the runtime path.

In a configured pinned Marin environment, the inspected CLI supports **printing a plan without submitting**:

```
uv run python -m experiments.grug.moe_hero_ep.launch_h100_scaling_ladder \
  --run-id replicate-d512 --size d512 --version 2026.11.02.1
```

`--run` would execute/build the artifacts; it is deliberately absent from this review command. This command has not been executed here. It still needs the adopted data store, validation artifacts, worker runtime and logging configuration. A prepared replacement corpus changes the reproduction contract. A short `--num-steps` override also changes horizon-derived HPs, so preserve the target optimizer explicitly when using a smoke test to validate the intended full-run numerics.

The 120-cell May main grid and 150-cell August grid are saved as specifications. They require exact model/active-count/branch choices before token counts or GPU costs can be derived; rounded issue-table parameter counts are not enough.

Our heavy-tuning plan: replicate first, then expand

Track A: a faithful Marin experiment. Pin either the May or August recipe, its optimizer groups, data/tokenizer, context, active-count formula and metric. Reproduce a small subset spanning at least two widths and three horizons with the **published five-point multiplier grids**. Confirm the local bowls before fitting coefficients. This tests replication; replacing the dataset/trainer with our own is Track B.

Track B: our corpus and 96-H100 configuration. Start a standard non-looped MoE, keeping recurrence and cyber upweighting separate. The search specification defines the following proposed staged search:

1. **Reference bracket:** 5 LR multipliers × 3 token batches (15 trials), with one frozen optimizer/schedule/corpus and fixed token horizon. Reject unstable ranges and expand boundary winners.
2. **Conditional regularization search:** for AdamW, five decay/timescale values near the selected neighborhood; for MuonH/AdamH, bracket scalar-group LR separately and verify norm axes instead of searching a nonexistent matrix

decay parameter.

3. **Adaptive refinement:** up to 40 Vizier suggestions, in ten rounds of four, over explicitly transformed log parameters and finite beta bounds. Compare against the best grid result using held-out terminal loss. Do not assume all seven dimensions need an exhaustive product.
4. **Width/horizon ladder:** three widths × three horizons × five LR multipliers = 45 runs. Horizons must include the user's 200+ stored-TPP regime, while also reporting active-TPP for MoE comparability.
5. **Batch law:** at three widths and two horizons, three token-batch brackets (18 runs) using the LR fit, then a joint neighborhood recheck. A fixed $\sqrt{B}$ exponent is not a batch optimum measurement.
6. **Fresh holdouts:** an unseen size/depth and a longer horizon, each with a local LR bracket and feasible adjacent batches; replicate final choices with fresh seeds. Compare tuned optimizers at equally annealed endpoints and equal total selection budgets.

This is a **heavier alternative** to the 80-run bounded queue, not additional free work. The old queue uses narrower LR brackets and a different experimental allocation. Price every candidate using measured per-shape throughput; prune or fund additional R&D before exceeding the 576-GPU-day preparation cap. A full historical 150-run sweep need not fit six calendar days even when its GPU-day sum fits.

Freeze warmup/decay shape for each law; record both token and update counts, all actual parameter-group settings, data order, phase boundaries and terminal evaluation coverage. New cyber data gets a transfer check at selected cells before a full retune. New recurrence or routing changes need their own qualified family.

Required experiment outputs

Every trial writes resolved config and source hashes; actual consumed tokens; parameter/group counts; full loss/gradient/norm/router trajectory; failure status; terminal annealed target and guardrail scores; hardware/throughput; best-tested versus interpolated optimum; and selection/holdout role. The optimizer fitter consumes these records, not hand-entered rounded summaries.

The next engineering step is to implement these records in the selected trainer and reproduce a few published cells. No tuning jobs have been run here, and no downloaded heuristic has been certified for our data or hardware.